P2077R2
Heterogeneous erasure overloads for associative containers

Published Proposal, 2020-12-15

This version:
http://wg21.link/P2077R2

Authors:
Konstantin Boyarinov (Intel)
Sergey Vinogradov (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The authors propose heterogeneous erasure overloads for ordered and unordered associative containers, which add an ability to erase values or extract nodes without creating a temporary `key_type` object.

Table of Contents

4 Performance evaluation

5 Formal wording

5.1 Modify [**tab:container.assoc.req**]

5.2 Modify paragraph **** in [**associative.reqmts.general**]

5.3 Modify class template `map` synopsis [**map.overview**]

5.4 Modify [**map.modifiers**]

5.5 Modify class template `multimap` synopsis [**multimap.overview**]

5.6 Modify [**multimap.modifiers**]

5.7 Modify class template `set` synopsis [**set.overview**]

5.8 Add paragraph **Modifiers** into [**set.overview**]

5.9 Modify class template `multiset` synopsis [**multiset.overview**]

5.10 Add paragraph **Modifiers** into [**multiset.overview**]

5.11 Modify [**tab:container.hash.req**]

5.12 Modify paragraph **** in [**unord.req.general**]

5.13 Modify class template `unordered_map` synopsis [**unord.map.overview**]

5.14 Modify [**unord.map.modifiers**]

5.15 Modify class template `unordered_multimap` synopsis [**unord.multimap.overview**]

5.16 Modify [**unord.multimap.modifiers**]

5.17 Modify [**unord.set.overview**] class template `unordered_set` synopsis

5.18 Add paragraph **Modifiers** into [**unord.set.overview**]

5.19 Modify class template `unordered_multiset` synopsis [**unord.multiset.overview**]

5.20 Add paragraph **Modifiers** into [**unord.multiset.overview**]

6 Revision history

6.1 R1 ➡ R2

6.2 R0 ➡ R1

References

Informative References

§ 1. Motivation

[N3657] and [P0919R0] introduced heterogeneous lookup support for ordered and unordered associative containers in C++ Standard Library. As a result, a temporary key object is not created when a different (but comparable) type is provided as a key to the member function. But there are no heterogeneous `erase` and `extract` overloads.

Consider the following example:

```
void foo( std::map<std::string, int, std::less<void>>& m ) {
    const char* lookup_str = "Lookup_str";
    auto it = m.find(lookup_str); // matches to template overload
    // some other actions with iterator
    m.erase(lookup_str); // causes implicit conversion
}
```

Function `foo` accepts a reference to the `std::map<std::string, int>` object. A comparator for the map is `std::less<void>`, which provides `is_transparent` valid qualified-id, so the `std::map` allows using heterogeneous overloads with `Key` template parameter for lookup functions, such as `find`, `upped_bound`, `equal_range`, etc.

In the example above, the `m.find(lookup_str)` call does not create a temporary object of the `std::string` to perform a lookup. But the `m.erase(lookup_str)` call causes implicit conversion from `const char*` to `std::string`. The allocation and construction (as well as subsequent destruction and deallocation) of the temporary object of `std::string` *or any custom object* can be quite expensive and reduce the program performance.

Erasure from the STL associative containers with the `key` instance of the type that is different from `key_type` is possible with the following code snippet:

```
auto eq_range = container.equal_range(key);
auto previous_size = container.size();
container.erase(eq_range.first, eq_range.second);
auto erased_count = container.size() - previous_size;
```

where `std::is_same_v<decltype(key), key_type>` is `false`.

`erased_count` determines the count of erased items and `container` is either:

- An ordered associative container in which `key_compare::is_transparent` is a valid qualified-id.

- An unordered associative container in which `hasher::is_transparent` and `key_equal::is_transparent` are valid qualified-ids.

The code above is a valid alternative for the heterogeneous `erase` overload. But heterogeneous `erase` would allow to do the same things more efficiently, without traversing the interval [`eq_range.first, eq_range.second`) twice (the first time to determine the equal range and the second time for erasure). It adds a performance penalty for the containers with non-unique keys (like `std::multimap`, `std::multiset`, etc.) where the number of elements with the same key can be quite large.

> Note: § 1 Motivation, § 3.1 Why K&& rather than const K& and § 4 Performance evaluation contain examples for the `erase` method. But the problems and benefits are similar for both `erase` and `extract` member functions.

§ 2. Prior Work

Possibility to add heterogeneous `erase` overload was reviewed in the [N3465]. But it was found that heterogeneous `erase` brakes backward compatibility and causes wrong overload resolution for the case when an `iterator` is passed as the argument. The `iterator` type is implicitly converted into `const_iterator` and the following overload of the erase method is called:

```
iterator erase( const_iterator pos )
```

If there was the following heterogeneous overload of the erase method:

```
template <class K>
size_type erase( const K& x );
```

template overload would be chosen in C++14 when an `iterator` object passed as the argument. So, it can cause the wrong effect or compilation error for legacy code.

C++17 introduces a new overload for `erase` method, which accepts exactly an object of `iterator` type as an argument:

```
iterator erase( iterator pos )
```

This change intended to fix the ambiguity issue [LWG2059] in the `erase` method overloads (for `key_type` and for `const_iterator`) when a `key_type` object can be constructed from the `iterator`.

Adding the overload with the `iterator` argument solves the problem described above but overload with template `K` parameter still will be chosen when passing an object of a type, which is implicitly convertible to either `iterator` or `const_iterator` that is not what user might expect.

§ 3. Proposal overview

We analyzed the prior work and basing on that we propose to add heterogeneous overloads for `erase` and `extract` methods in `std::map`, `std::multimap`, `std::set`, `std::multiset`, `std::unordered_map`, `std::unordered_multimap`, `std::unordered_set` and `std::unordered_multiset`:

```
template <class K>
size_type erase( K&& x );
```

and

```
template <class K>
node_type extract( K&& x );
```

To maintain backward compatibility and avoid wrong overload resolution or compilation errors these overloads should impose extra restrictions on the type `K`.

For ordered associative containers these overloads should participate in overload resolution only if all the following statements are true:

1. Qualified-id `Compare::is_transparent` is valid and denotes a type.
2. The type `K&&` is not convertible to the `iterator`.
3. The type `K&&` is not convertible to the `const_iterator`.

where `Compare` is a type of comparator passed to an ordered container.

For unordered associative containers these overloads should participate in overload resolution if all the following statements are true:

1. Qualified-id `Hash::is_transparent` is valid and denotes a type.

2. Qualified-id `Pred::is_transparent` is valid and denotes a type.
3. The type `K&&` is not convertible to the `iterator`.
4. The type `K&&` is not convertible to the `const_iterator`.

where `Hash` and `Pred` are types of hash function and key equality predicate passed to an unordered container respectively.

§ 3.1. Why `K&&` rather than `const K&`

Initially we considered making an interface for heterogeneous erasure methods consistent with heterogeneous lookup:

```
template <class K>
size_type erase( const K& x );
```

and adding the following constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type.
  For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types and
- The type `K` is not convertible to the `iterator` and
- The type `K` is not convertible to the `const_iterator`.

The feedback from LEWG was to investigate convertability of all value categories of `K` (`K&`, `const K&`, `K&&`, `const K&&`) to `iterator` or `const_iterator`, similar to what `std::optional` does for constructors.

During the investigation we defined the heterogeneous erasuse with the `const K&` function parameter:

```
template <class K>
size_type erase( const K& x );
```

with the following constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type.

For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types and

- The type `K&` is not convertible to either `iterator` and `const_iterator` and
- The type `const K&` is not convertible to either `iterator` and `const_iterator` and
- The type `K&&` is not convertible to either `iterator` and `const_iterator` and
- The type `const K&&` is not convertible to either `iterator` and `const_iterator`.

Unfortunately, with that definition we discovered the following issue.

Consider `map_type` as `std::map` for which `Compare::is_transparent` is valid and denotes a type.

```
struct HeterogeneousKey {
    HeterogeneousKey() { /*...*/ }
    operator map_type::iterator() && { /*Some conversion logic*/ }

    // Comparison operations with map_type::key_type
};

void foo() {
    map_type m;
    HeterogeneousKey key;

    m.erase(key);
}
```

The code above fails to compile because:

1. For heterogeneous erasure overload, the type `K` is deduced as `HeterogeneousKey`, for which `std::is_convertible_v<HeterogeneousKey&&, iterator>` is `true` and hence the heterogeneous erasure does not participate in overload resolution.
2. An object `key` passed to `erase` method cannot be converted to the `iterator` due to rvalue ref-qualifier for the conversion operator.

Therefore, the matching overload for the call `m.erase(key)` is not found.

The heterogeneous erasure overload should participate in overload resolution when non-template overloads is not matched. With the current API we lose the information about the value category of `K` due to template argument deduction from `const K&` function parameter. Therefore, we cannot define constraints over `K` for the arbitrary case.

To propagate the information about the value category of `K` we define the function parameter for heterogeneous erasure as forwarding reference. The interface is

```
template <class K>
size_type erase( K&& x );
```

with the following constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type.
  For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types and
- The type `K&&` is not convertible to the `iterator`.
- The type `K&&` is not convertible to the `const_iterator`.

This overload does not participate in overload resolution only if the function argument of the particular value category is convertible to the `iterator` or `const_iterator`. The example above is compiled and the heterogeneous `erase` overload is called for `m.erase(key)` line.

> Note: the example is shown for `std::map` but the issue is relevant for all associative containers.

§ 4. Performance evaluation

We estimated the performance impact on two synthetic benchmarks:

- Erase all elements consistently from the `std::unordered_map<std::string, int>`, filled by 10000 values with unique keys. Size of each `std::string` key is 1000.
- Erase all elements from the `std::unordered_multimap<std::string, int>` filled by 10000 values with duplicated keys. Size of each `std::string` key is 1000.

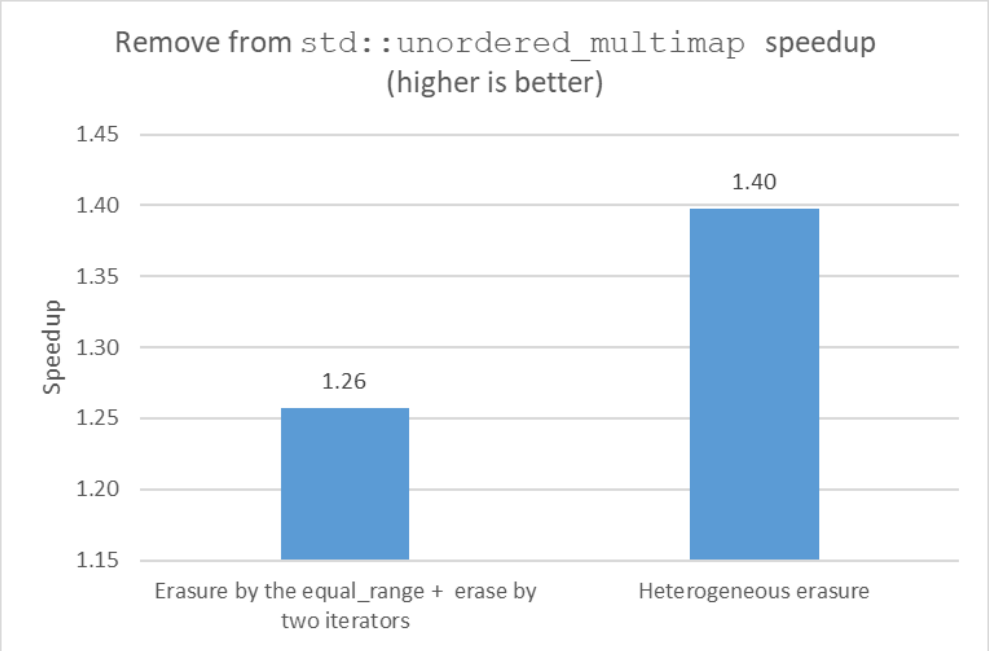
To do that we have implemented heterogeneous `erase` method for `std::unordered_map` and `std::unordered_multimap` on the base of LLVM Standard Library implementation.

We have compared the performance of three possible erasure algorithms:

- Erasure by `key_type` object.
- Erasure by the pair of iterators, obtained by the heterogeneous `equal_range`.
- Heterogeneous erasure with the `std::string_view` as an argument.

The benchmark for `std::unordered_map` shows that the erasure by the pair of iterators (as well as heterogeneous erasure) increases performance by ~20%.

The benchmark for `std::unordered_multimap` shows the same performance increase for erasure by the pair of iterators and an additional performance increase by ~10% for heterogeneous erasure (due to double traversal of `equal_range`).



To do the additional analysis with different memory allocation source we took an open-source application *pmemkv*. It is an embedded key/value data storage designed for emergent persistent memory. *pmemkv* has several storage engines optimized for different use cases. For the analysis we

chose *vsmap* engine that is built on `std::map` data structure with allocator for persistent memory from the *memkind library*. `std::basic_string` with the *memkind* allocator was used as a key and value type.

```
using storage_type = std::basic_string<char, std::char_traits<char>,
                                       libmemkind::pmem::allocator<char>>;
using key_type = storage_type;
using mapped_type = storage_type;
using map_allocator_type = libmemkind::pmem::allocator<std::pair<const key_
                                                                 mapped_typ
using map_type = std::map<key_type, mapped_type, std::less<void>,
                          std::scoped_allocator_adaptor<map_allocator_type>
```

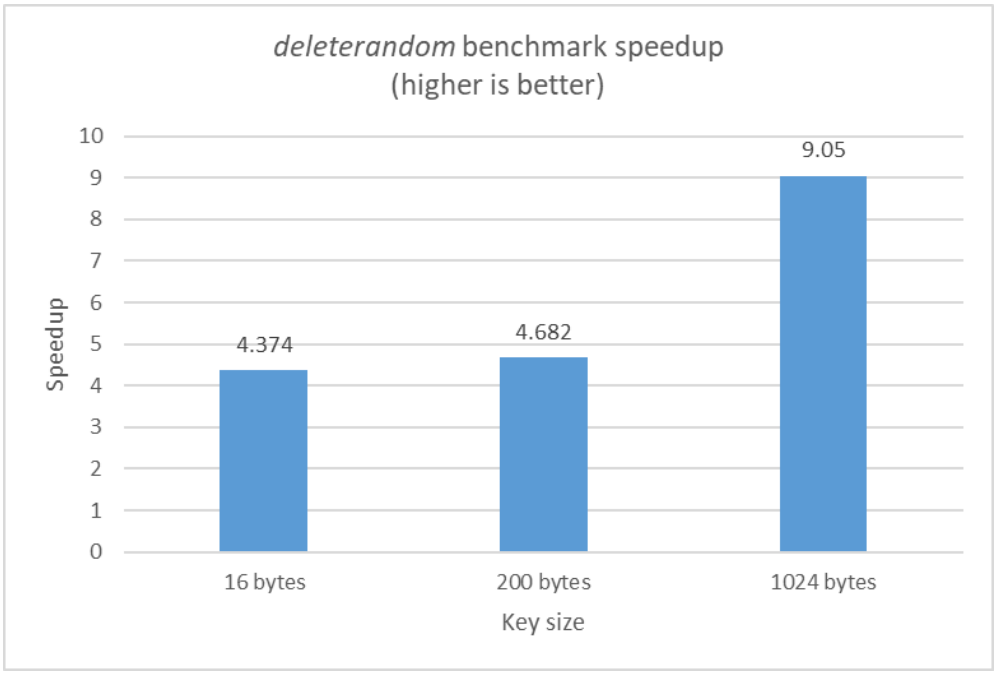
Initial implementation of `remove` method of *vsmap* engine was the following:

```
status vsmap::remove( std::string_view key ) {
    std::size_t erased = c.erase(key_type(key.data(), key.size(), a));
    return (erased == 1) ? status::OK : status::NOT_FOUND;
}
```

The initial implementation explicitly creates temporary object of `key_type` when `erase` method is called. To estimate performance impact of the heterogeneous `erase` overload we re-designed `remove` operation of the *vsmap* engine in the following way:

```
status vsmap::remove( std::string_view key ) {
    auto it = c.find(key);
    if (it != c.end()) {
        c.erase(it);
        return status::OK;
    }
    return status::NOT_FOUND;
}
```

To understand the performance impact we used *pmemkv utility*. We run *deleterandom* benchmark on prefilled storage and measured throughput as a number of operations per second. We executed the test with different key sizes (16 bytes, 200 bytes, 1024 bytes). The chart below shows performance increase, comparing to initial implementation, for all tests. The throughput of the `remove` operation increased up to 9x for the 1024 bytes key.



§ 5. Formal wording

Below, substitute the ﬠ character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

§ 5.1. Modify [**tab:container.assoc.req**]

Table **[tab:container.assoc.req]**: **Associative container requirements (in addition to container)**

Expression	Return type	Assertion/ note/ pre-/ post-condition	Complexity
[...]			
`a.extract(k)`	`node_type`	*Effects*: Removes the first element in the container with key equivalent to `k`. *Returns*: A `node_type` owning the element if found, otherwise an empty `node_type`.	log(`a.size()`)
`a_tran.extract(ke)`	`node_type`	*Effects*: Removes the first element in the container with key `r` such that `!c(r, ke) && !c(ke, r)`. *Returns*: A `node_type` owning the element if found, otherwise an empty `node_type`.	log(`a_tran.size()`)
[...]			
`a.erase(k)`	`size_type`	*Effects*: Erases all elements in the container with key equivalent to `k`. *Returns*: The number of erased elements.	log(`a.size()`) + `a.count(k)`
`a_tran.erase(ke)`	`size_type`	*Effects*: Erases all elements in the container with key `r` such that `!c(r, ke) && !c(ke, r)`. *Returns*: The number of erased elements.	log(`a_tran.size()`) + `a_tran.count(ke)`
[...]			

§ 5.2. Modify paragraph 🔲 in [**associative.reqmts.general**]

> [...]
> The member function templates `find`, `count`, `contains`, `lower_bound`, `upper_bound`, ~~and~~ `equal_range` , `erase`, and `extract` shall not participate in overload resolution unless the *qualified-id* `Compare::is_transparent` is valid and denotes a type (13.10.3).
> [...]

§ 5.3. Modify class template `map` synopsis [**map.overview**]

> [...]
>
> ```
> node_type extract(const_iterator position);
> node_type extract(const key_type& x);
>
> template<class K> node_type extract(K&& x);
> ```
>
> [...]
>
> ```
> iterator erase(iterator position);
> iterator erase(const_iterator position);
> size_type erase(const key_type& x);
>
> template<class K> size_type erase(K&& x);
> ```
>
> [...]

§ 5.4. Modify [**map.modifiers**]

```
  template<class M>
    pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
  template<class M>
    iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
```

[...]

— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K> size_type erase(K&& key);
```

— *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`

```
  template<class K> node_type extract(K&& key);
```

— *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`

§ 5.5. Modify class template `multimap` synopsis [**multimap.overview**]

[...]

```
node_type extract(const_iterator position);
node_type extract(const key_type& x);

template <class K> node_type extract(K&& x);
```

[...]

```
iterator erase(iterator position);
iterator erase(const_iterator position);
size_type erase(const key_type& x);

template <class K> size_type erase(K&& x);
```

[...]

§ 5.6. Modify [**multimap.modifiers**]

```
template<class P> iterator insert(P&& x);
template<class P> iterator insert(const_iterator position, P&& x);
```

- *Constraints*: `is_constructible_v<value_type, P&&>` is `true`.
- *Effects*: The first form is equivalent to `return emplace(std::forward<P>(x)`
  The second form is equivalent to `return emplace_hint(position, std::forward<`

```
template<class K> size_type erase(K&& key);
```

- *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`

```
template<class K> node_type extract(K&& key);
```

- *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`

§ 5.7. Modify class template `set` synopsis [**set.overview**]

[...]

```
node_type extract(const_iterator position);
node_type extract(const key_type& x);

template <class K> node_type extract(K&& x);
```

[...]

```
iterator erase(iterator position);
iterator erase(const_iterator position);
size_type erase(const key_type& x);

template <class K> size_type erase(K&& x);
```

[...]

§ 5.8. Add paragraph **Modifiers** into [**set.overview**]

— Erasure [set.erasure]

[...]

Modifiers [set.modifiers]

template<class K> size_type erase(K&& key);

— *Constraints*: is_convertible_v<K&&, iterator> and is_convertible_v<K&&, const_iterator> are false

template<class K> node_type extract(K&& key);

— *Constraints*: is_convertible_v<K&&, iterator> and is_convertible_v<K&&, const_iterator> are false

§ 5.9. Modify class template `multiset` synopsis [**multiset.overview**]

[...]

```
node_type extract(const_iterator position);
node_type extract(const key_type& x);

template <class K> node_type extract(K&& x);
```

[...]

```
iterator erase(iterator position);
iterator erase(const_iterator position);
size_type erase(const key_type& x);

template <class K> size_type erase(K&& x);
```

[...]

§ 5.10. Add paragraph **Modifiers** into [**multiset.overview**]

> **Erasure [multiset.erasure]**
>
> [...]
>
> **Modifiers [multiset.modifiers]**
>
> `template<class K> size_type erase(K&& key);`
>
> — *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`
>
> `template<class K> node_type extract(K&& key);`
>
> — *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`

§ 5.11. Modify [**tab:container.hash.req**]

Table **[tab:container.hash.req]**: **Unordered associative container requirements (in addition to container)**

Expression	Return type	Assertion/ note/ pre-/ post-condition	Complexity
[...]			
`a.extract(k)`	`node_type`	*Effects*: Removes an element in the container with key equivalent to `k`. *Returns*: A `node_type` owning the element if found, otherwise an empty `node_type`.	Average case O(1), worst case O(`a.size()`).
`a_tran.extract(ke)`	`node_type`	*Effects*: Removes an element in the container with key equivalent to `ke`. *Returns*: A `node_type` owning the element if found, otherwise an empty `node_type`.	Average case O(1), worst case O(`a_tran.size()`).
[...]			
`a.erase(k)`	`size_type`	*Effects*: Erases all elements with key equivalent to `k`. *Returns*: The number of elements erased.	Average case O(`a.count(k)`), worst case O(`a.size()`).
`a_tran.erase(ke)`	`size_type`	*Effects*: Erases all elements with key equivalent to `ke`. *Returns*: The number of elements erased.	Average case O(`a_tran.count(k)`), worst case O(`a_tran.size()`).
[...]			

§ 5.12. Modify paragraph 🔲 in [**unord.req.general**]

> [...]
> The member function templates `find`, `count`, `equal_range`, ~~and~~ `contains` , `erase`, and `extract` shall not participate in overload resolution unless the *qualified-ids* `Pred::is_transparent` and `Hash::is_transparent` are both valid and denote types (13.10.3).
> [...]

§ 5.13. Modify class template `unordered_map` synopsis [**unord.map.overview**]

> [...]
>
> ```
> node_type extract(const_iterator position);
> node_type extract(const key_type& x);
>
> template <class K> node_type extract(K&& x);
> ```
>
> [...]
>
> ```
> iterator erase(iterator position);
> iterator erase(const_iterator position);
> size_type erase(const key_type& x);
>
> template <class K> size_type erase(K&& x);
> ```
>
> [...]

§ 5.14. Modify [**unord.map.modifiers**]

```
  template<class M>
    pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
  template<class M>
    iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
```

[...]

- *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K> size_type erase(K&& key);
```

- *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`

```
  template<class K> node_type extract(K&& key);
```

- *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`

§ 5.15. Modify class template `unordered_multimap` synopsis [**unord.multimap.overview**]

[...]

```
node_type extract(const_iterator position);
node_type extract(const key_type& x);

template <class K> node_type extract(K&& x);
```

[...]

```
iterator erase(iterator position);
iterator erase(const_iterator position);
size_type erase(const key_type& x);

template <class K> size_type erase(K&& x);
```

[...]

§ 5.16. Modify [**unord.multimap.modifiers**]

```
template<class P> iterator insert(const_iterator position, P&& x);
```

- *Constraints*: `is_constructible_v<value_type, P&&>` is `true`.
- *Effects*: Equivalent to: `return emplace_hint(position, std::forward<P>(x))`.

```
template<class K> size_type erase(K&& key);
```

- *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`

```
template<class K> node_type extract(K&& key);
```

- *Constraints*: `is_convertible_v<K&&, iterator>` and `is_convertible_v<K&&, const_iterator>` are `false`

§ 5.17. Modify [**unord.set.overview**] class template `unordered_set` synopsis

[...]

```
node_type extract(const_iterator position);
node_type extract(const key_type& x);

template <class K> node_type extract(K&& x);
```

[...]

```
iterator erase(iterator position);
iterator erase(const_iterator position);
size_type erase(const key_type& x);

template <class K> size_type erase(K&& x);
```

[...]

§ 5.18. Add paragraph **Modifiers** into [**unord.set.overview**]

> **Erasure [unord.set.erasure]**
>
> [...]
>
> **Modifiers [unord.set.modifiers]**
>
> **template<class K> size_type erase(K&& key);**
>
> *Constraints*: is_convertible_v<K&&, iterator> and is_convertible_v<K&&, const_iterator> are false
>
> **template<class K> node_type extract(K&& key);**
>
> *Constraints*: is_convertible_v<K&&, iterator> and is_convertible_v<K&&, const_iterator> are false

§ 5.19. Modify class template `unordered_multiset` synopsis [**unord.multiset.overview**]

> [...]
>
> ```
> node_type extract(const_iterator position);
> node_type extract(const key_type& x);
>
> template <class K> node_type extract(K&& x);
> ```
>
> [...]
>
> ```
> iterator erase(iterator position);
> iterator erase(const_iterator position);
> size_type erase(const key_type& x);
>
> template <class K> size_type erase(K&& x);
> ```
>
> [...]

§ 5.20. Add paragraph **Modifiers** into [**unord.multiset.overview**]

> **Erasure [unord.multiset.erasure]**
>
> [...]
>
> **Modifiers [unord.multiset.modifiers]**
>
> **template<class K> size_type erase(K&& key);**
>
> *Constraints*: is_convertible_v<K&&, iterator> and is_convertible_v<K&&, const_iterator> are false
>
> **template<class K> node_type extract(K&& key);**
>
> *Constraints*: is_convertible_v<K&&, iterator> and is_convertible_v<K&&, const_iterator> are false

§ 6. Revision history

§ 6.1. R1 ➡ R2

- Changed interface of heterogeneous erasure overloads from `const K&` to `K&&`. For more information see § 3.1 Why K&& rather than const K&.
- Aligned wording with the standard (fonts, capital letters, etc.)

§ 6.2. R0 ➡ R1

Addressed feedback from the presentation on LEWG-I to align wording and approach with [P1690R1].

§ References

§ Informative References

[LWG2059]

Christopher Jefferson. C++0x ambiguity problem with map::erase. C++17. URL: https://wg21.link/lwg2059

[N3465]

Joaquín Mª López Muñoz. Adding heterogeneous comparison lookup to associative containers for TR2 (Rev 2). 29 October 2012. URL: https://wg21.link/n3465

[N3657]

J. Wakely, S. Lavavej, J. Muñoz. Adding heterogeneous comparison lookup to associative containers (rev 4). 19 March 2013. URL: https://wg21.link/n3657

[P0919R0]

Mateusz Pusz. Heterogeneous lookup for unordered containers. 8 February 2018. URL: https://wg21.link/p0919r0

[P1690R1]

Xiao Shi, Mateusz Pusz, Geoffrey Romer. Refinement Proposal for P0919 Heterogeneous lookup for unordered containers. 12 August 2019. URL: https://wg21.link/p1690r1